

Phase 0 — Explained

Project setup & foundations · every file, every decision, every next step

22 July 2026 · Commit `b9315c4` · 61 files · Phase 0 of 11

Contents

1. What Phase 0 was for
2. The complete file map
3. Repository-level files explained
4. The Python agent explained
5. The dashboard explained
6. The documentation tool
7. Key concepts (read this if anything above was unclear)
8. Decisions I made, and why
9. Problems hit during the build, and how they were fixed
10. What you should do right now — step by step
11. What you need before each future phase
12. Open questions I need answers to

1. What Phase 0 was for

Phase 0 builds **no features**. Its only job is to create a clean, running skeleton so that from Phase 1 onward every piece of real work has somewhere obvious to go. The build plan defines the goal as: *"a clean, running skeleton for both the agent and the dashboard."*

Concretely, the phase is complete when four things are true:

- The folder structure matches `03_PROJECT_STRUCTURE.md` exactly.
- The Python agent runs and reports its own configuration state.
- The React dashboard builds and displays a page.
- Every environment variable the project will ever need is documented in one place.

All four are verified done. The agent was executed and printed its self-check; the dashboard compiled to a production build in 4.26 seconds with zero errors. Neither does real work yet — that is correct and expected at this stage.

The one thing added beyond the plan

You asked that nothing from the spec be missed. A promise is not checkable, so I created

`REQUIREMENTS_COVERAGE.md` — a ledger giving every single requirement an ID and mapping it to the phase that delivers it. A phase cannot be closed until all of its rows are marked done. This is the mechanism that makes "nothing missed" a fact rather than a hope.

2. The complete file map

61 files are tracked in git. Ignoring the 23 documentation stubs (listed separately in section 4), here is everything created or edited:

File	Status	What it is
<code>.gitignore</code>	NEW	Stops secrets and build junk being committed
<code>.env.example</code>	NEW	Every environment variable, annotated with the phase that needs it
<code>README.md</code>	NEW	What the project is and how to run both halves
<code>PROGRESS.md</code>	NEW	Phase tracker + decisions log; how a fresh session resumes
<code>REQUIREMENTS_COVERAGE.md</code>	NEW	Every requirement mapped to a phase
<code>database/schema.sql</code>	NEW	Reference copy of the Supabase schema
<code>agent/requirements.txt</code>	EDITED	Python dependencies
<code>agent/config.py</code>	NEW	The single place all secrets and settings are read
<code>agent/db/client.py</code>	NEW	Supabase connection for the agent
<code>agent/main.py</code>	EDITED	Entry point; currently a setup self-check
<code>agent/**/*.py</code> (9)	NEW	Makes each folder an importable Python package
<code>agent/ stubs</code> (23)	NEW	Placeholder modules documenting future work
<code>dashboard/package.json</code>	NEW	Dashboard dependencies and scripts
<code>dashboard/vite.config.js</code>	EDITED	Build config; React + Tailwind + network access
<code>dashboard/index.html</code>	NEW	Page shell + the iPhone PWA meta tags
<code>dashboard/src/main.jsx</code>	NEW	Mounts React into the page
<code>dashboard/src/App.jsx</code>	NEW	The Phase 0 status page
<code>dashboard/src/index.css</code>	NEW	Tailwind import, temperature colours, iPhone safe areas
<code>dashboard/src/lib/supabase.js</code>	NEW	Supabase connection for the browser
<code>dashboard/public/manifest.json</code>	NEW	What makes it installable on a phone
<code>dashboard/public/icons/</code> (4)	NEW	App icons: 180/192/512 PNG + SVG favicon
<code>tools/make_pdf.py</code>	NEW	Turns these phase documents into PDFs
<code>docs/PHASE_0_EXPLAINED.html</code>	NEW	The source of the document you are reading

Not in git, correctly: `agent/venv/` (your Python packages), `dashboard/node_modules/` (92 JavaScript packages), `dashboard/dist/` (the compiled output). These are regenerated from `requirements.txt` and `package.json`, so committing them would only bloat the repository.

3. Repository-level files explained

`.gitignore`

What it does: tells git which files to never track.

Why it was the very first file written: your repository is *public*. Once a secret is committed and pushed, deleting it later does not remove it — it stays in the git history forever, and bots scan public GitHub for exactly this. Writing `.gitignore` before the first `git add` means an `.env` file can never be committed by accident.

What it blocks: all `.env` variants, `*.pem` and `*.key`, `venv/`, `node_modules/`, `__pycache__/`, `dist/`, Playwright browser profiles (which will contain your logged-in LinkedIn and Instagram sessions), and editor/OS clutter.

Verified: after committing, I ran a check confirming no `.env` file is tracked.

`.env.example`

What it does: documents every environment variable the project will ever need, with placeholder values instead of real secrets. It is committed; the real `.env` files are not.

Why it matters: it is the answer to "what do I need to fill in?" Each block carries a comment saying which phase actually requires it, so you can see that an empty Claude API key is fine now but blocks Phase 4.

Structure: two sections. The first is for `agent/.env` (Supabase URL and *service role* key, Claude key, model names, WhatsApp credentials, timezone, headless flag, a dev safety cap). The second is for `dashboard/.env` (Supabase URL and *anon* key only).

Note it contains: proxy credentials are deliberately *not* here. They live per-account in the Supabase `accounts` table so you can edit them from the dashboard — which is what makes "add proxies later" a settings change with zero code change, exactly as the spec requires.

REQUIREMENTS_COVERAGE.md

What it does: the anti-omission ledger. Eight tables of requirements, each row with an ID, the requirement, the file that implements it, the phase, and a status.

Group	Count	Contents
A — Core rules	10	R1–R10, the never-violate constraints
B — Agent tasks	17	T1–T17 from spec §6, in execution order
C — Dashboard	11	D1–D11, every screen plus the PWA
D — Settings	15	S1–S15, every editable control
E — Notifications	5	N1–N5, the WhatsApp alert types
F — Safety/proxy	7	P1–P7, proxy rules and warm-up
G — Out of scope	11	X1–X11, listed so they are never accidentally built
H — Open questions	7	Q1–Q7, decisions I need from you

How it gets used: at the end of every phase I mark that phase's rows `DONE`. If a row cannot be marked, the phase does not close. Group G exists because the spec explicitly lists features that were *considered and cut* — writing them down prevents accidentally building something you decided against.

PROGRESS.md

What it does: the phase tracker required by the master prompt. Shows the status of all 11 phases, a full write-up of what Phase 0 built, how to test it, decisions made, and what is still open.

Why it exists: if this chat session runs out of memory, a fresh session can read `PROGRESS.md` plus `00_MASTER_PROMPT.md` and continue without you re-explaining anything.

README.md

What it does: the front door of the repository. Explains what the system is in one paragraph, states the three channel rules in a table (LinkedIn automated, WhatsApp conditional, Instagram never automated), gives the exact commands to run both halves, shows the folder layout, and links every reference document.

database/schema.sql

What it does: a copy of `02_SUPABASE_SCHEMA.sql`, placed where `03_PROJECT_STRUCTURE.md` asks for it. Reference only — the real schema gets run inside Supabase in Phase 1.

4. The Python agent explained

`agent/requirements.txt`

What it does: lists the Python packages the agent needs, so the same environment can be rebuilt on your laptop and on the server with one command.

Package	Why it's there
supabase	Reads and writes the database
anthropic	Calls Claude — Haiku for analysis, Sonnet for messages
playwright	Drives a real browser for LinkedIn/Instagram discovery and LinkedIn sending
python-dotenv	Loads <code>agent/.env</code> into the process
APScheduler	Runs each account at its own configured time
httpx	HTTP calls — website analysis, WhatsApp API
beautifulsoup4	Parses lead websites to find weak points in Phase 4
pytz	Timezone handling for per-account run times

Edited during the build: the versions were originally pinned exactly (`==`). That combination was impossible to install — see section 9, problem 1.

agent/config.py

What it does: reads every secret and setting from `agent/.env`, and is the *only* file that does so.

Why one file: when something is misconfigured there is exactly one place to look. Nothing else in the codebase touches environment variables directly.

The pieces:

- `_get()` — reads a variable and treats an empty string as missing. Without this, a line like `ANTHROPIC_API_KEY=` would count as "set" and you would get a confusing authentication failure later instead of a clear message now.
- Supabase URL and **service role** key.
- Claude API key, plus the two model names as variables so the models can be changed without editing code.
- WhatsApp provider settings — defaulted to Twilio, but this is open question Q1.
- `TIMEZONE` (default `Asia/Beirut`) — how per-account run times are interpreted.
- `HEADLESS` — when false, Playwright shows a visible browser window. You will want that while we build discovery in Phase 3 so you can watch it work.
- `DEV_MAX_LEADS_PER_ACCOUNT` — a safety brake that caps volume during local testing regardless of the dashboard setting. Protects your accounts from an accidental full-volume run during development.
- `missing_required()` — each phase asks only for the keys it needs, so Phase 0 runs happily with a totally empty `.env` while Phase 4 can refuse to start without a Claude key.

agent/db/client.py

What it does: creates and shares the Supabase connection.

- **Lazy creation.** The connection is made on first use, not on import. This is why the agent runs today with no credentials at all — importing the module cannot fail.
- **One shared client.** Creating a client opens a connection pool, so the module caches a single instance rather than making a new one per call.
- **Service role key.** The agent runs on a server you control and needs full access to every table. See section 7 for why this key must never appear in the dashboard.
- `check_connection()` — proves the credentials actually work by selecting one row from `settings`. It returns `(ok, message)` instead of raising, because in the Phase 0 self-check a failure is information, not a crash.

agent/main.py

What it does today: runs a self-check and prints which parts of the system are configured, with the phase each missing piece is needed for.

What it becomes: from Phase 2 this is the daily run loop — for each account, at its own scheduled time, open a session and work through Tasks 1 to 17.

Run it with:

```
agent/venv/Scripts/python.exe -m agent.main
```

Current output:

```
=====
NEXARIS AI OUTREACH AGENT - setup self-check
=====

[ok] Python runtime           Python 3.11.9
[--] Supabase credentials    not set - needed for Phase 1
[--] Claude API key          not set - needed for Phase 4
[--] WhatsApp credentials    not set - needed for Phase 7

Models:   analysis=claude-haiku-4-5-20251001
          messages=claude-sonnet-5
Timezone: Asia/Beirut
Headless: True
```

Three `[--]` marks is the *correct* result right now. They are a to-do list, not errors.

The 9 `__init__.py` files

A folder only becomes an importable Python package when it contains an `__init__.py`. Without them, `from agent.db import client` fails. Each one holds a single line describing what its folder is for.

The 23 stub modules

Every file named in `03_PROJECT_STRUCTURE.md` now exists, containing a docstring that states what it will do, which phase builds it, and the specific rules it must obey. They contain no code yet.

Why create empty files? Three reasons. First, the structure is real and navigable now rather than theoretical. Second, each stub carries the rules with it — for example `sending/instagram_queue.py` opens with "THIS MODULE NEVER SENDS", so the constraint sits in the file where it applies rather than only in a document. Third, when we reach Phase 3 there is no ambiguity about where LinkedIn discovery goes.

Module	Phase	Purpose
<code>scheduler.py</code>	2	Runs each account at its own time; opens and closes the <code>runs</code> row
<code>core/account_pool.py</code>	2	Loads the 3 accounts, selects by run time
<code>core/session.py</code>	2	One isolated browser context per account, on its own proxy
<code>core/health.py</code>	2	Detects problems, records warning type + specific reason
<code>core/warmup.py</code>	2	Ramps daily volume from 5–10 up to full over weeks
<code>discovery/linkedin.py</code>	3	LinkedIn search and profile reading, 30/account
<code>discovery/instagram.py</code>	3	Instagram search and reading, 20/account, read-only
<code>discovery/qualify.py</code>	3	Is this a real business worth pursuing?
<code>analysis/analyze.py</code>	4	Deep company analysis via Haiku
<code>analysis/founder.py</code>	4	Founder detection from bios
<code>analysis/whatsapp_detect.py</code>	4	Finds public WhatsApp numbers
<code>analysis/score.py</code>	4	Scores 1–10 with written reasoning
<code>analysis/prompts.py</code>	4	All Claude prompts, structured for caching
<code>messaging/generate.py</code>	5	Message generation via Sonnet
<code>messaging/style.py</code>	5	Direct/Discovery style and rotation
<code>sending/linkedin_send.py</code>	7	Auto-send on LinkedIn
<code>sending/whatsapp_send.py</code>	7	Auto-send on WhatsApp
<code>sending/instagram_queue.py</code>	7	Manual-send queue — never sends
<code>crm/pipeline.py</code>	8	Stage movement, timestamped history
<code>crm/followup.py</code>	8	Follow-up, re-engagement, 2-contact maximum
<code>crm/reply_detection.py</code>	8	Detects replies, advances the pipeline
<code>notifications/whatsapp_notify.py</code>	8	The 5 alert types
<code>db/repositories.py</code>	1	Read/write functions per table

5. The dashboard explained

dashboard/package.json

What it does: lists JavaScript dependencies and the commands to run.

Dependencies: React 18, React DOM, React Router (for navigating between the 10 sections in Phase 9), and the Supabase JavaScript client. **Build tools:** Vite 6, the React plugin, and Tailwind 4.

Commands: `npm run dev` (development server), `npm run build` (production build for Vercel), `npm run preview` (test the production build locally).

dashboard/vite.config.js

What it does: configures the build. Three settings, each deliberate:

- **Plugins** — React plus Tailwind v4. In v4 Tailwind is a Vite plugin, which is why there is no `tailwind.config.js`.
- `css.postcss: {}` — added mid-build to fix a real failure. See section 9, problem 3.
- `server.host: true` — exposes the dev server on your local network so you can open the dashboard on your *phone* during development, not just the laptop. Given that Mohamad will use this on an iPhone, testing the mobile layout early matters.

dashboard/index.html

What it does: the page shell React mounts into. Most of it is the PWA setup that makes "Add to Home Screen" work properly on Mohamad's iPhone:

- `viewport-fit=cover` — lets the app draw into the notch area.
- `apple-mobile-web-app-capable` — this is the tag that makes iOS open it *full-screen with no Safari address bar*. Without it, "Add to Home Screen" produces a bookmark rather than something that feels like an app.
- `apple-mobile-web-app-title` — the name under the home screen icon.
- `apple-touch-icon` — the icon itself.
- `link rel="manifest"` — the manifest below.

dashboard/public/manifest.json + icons/

What it does: declares the app's name, colours, icons, and that it should launch `standalone` (full-screen, no browser chrome).

The icons are 180px (iOS home screen), 192px and 512px (the manifest sizes), plus an SVG favicon. I generated them programmatically — a dark slate square with a light "N" — using a small pure-Python PNG encoder, so no image library was needed. *These are placeholders*. Replace them with real Nexaris branding in Phase 9.

dashboard/src/lib/supabase.js

What it does: the browser's database connection.

- Uses the **anon key** — never the service role key. See section 7.
- Exports `isConfigured` so the UI can show a helpful setup message instead of crashing when `.env` is empty, which is the state right now.
- Enables **Realtime**, which is what will make the live feed, run status, and analytics update by themselves as the agent works — no refresh, no polling.

dashboard/src/index.css

What it does: imports Tailwind and sets three things.

- **Temperature colours** — hot, warm, and cold defined once under `@theme`. The Hot/Warm/Cold badge appears on live feed cards, pipeline cards, the approval queue and client history; defining the colours in one place means every screen agrees on what "hot" looks like.
- **Safe-area padding** — keeps content clear of the iPhone notch and home indicator in full-screen mode.
- `-webkit-text-size-adjust` — stops iOS Safari auto-enlarging text in landscape.

dashboard/src/main.jsx and src/App.jsx

`main.jsx` mounts React into the page — four lines, standard.

`App.jsx` is the Phase 0 status page. It shows whether the dashboard built correctly and whether Supabase credentials are present, then lists the 10 sections coming in Phase 9 so the build order is visible. It is written responsively (`max-w-3xl`, `sm:` breakpoints) because every screen in this project has to work on both a laptop and a phone.

Verified: production build succeeded — 70 modules, 4.26 seconds, 10 KB of CSS and 362 KB of JavaScript (103 KB gzipped).

6. The documentation tool

tools/make_pdf.py

What it does: turns a phase document from HTML into PDF.

How: it drives the Chromium browser that Playwright already installs for the agent, loads the HTML, and prints it to A4 with page numbers. Because Playwright is already a dependency, this adds nothing new to install.

Usage:

```
agent/venv/Scripts/python.exe tools/make_pdf.py docs/PHASE_0_EXPLAINED.html
```

Every phase from here on will produce one of these.

7. Key concepts

The two Supabase keys — the most important thing on this page

Supabase gives you two keys and they are **not** interchangeable.

Key	Used by	Behaviour
anon	Dashboard	Safe to ship in browser code. It is restricted by Row Level Security — the rules we write in Phase 1 that say only you and Mohamad may read or write.
service role	Agent	Bypasses Row Level Security entirely. Full read/write on every table. Safe only on a server you control.

If the service role key ever ends up in the dashboard, it gets compiled into the JavaScript that every visitor downloads — and since your repository is public, anyone could take it and read or delete your entire database. This is why the two clients are separate files reading separate variables, and why `.env` is gitignored. Whenever you copy keys, check twice which one you are pasting.

Why a virtual environment (`venv`)

A `venv` is a private Python installation for this project only. Its packages cannot conflict with other Python work on your machine, and `requirements.txt` lets the server rebuild the identical environment. That is why commands use `agent/venv/Scripts/python.exe` rather than plain `python`.

What "PWA" actually buys you

A Progressive Web App is a website that can be installed. On Mohamad's iPhone: open in Safari, Share, Add to Home Screen — it then gets an icon and opens full-screen with no address bar. No App Store, no Apple developer account, no separate iOS codebase. One React app serves your laptop and his phone.

Why the agent and dashboard are separate

They share one Supabase database but are otherwise independent. The agent writes what it discovers; the dashboard reads it and writes back settings, approvals, and manual actions. They are deployed to different places (agent to an always-on server, dashboard to Vercel) and can be restarted independently — a dashboard deploy never interrupts a run.

8. Decisions I made, and why

Decision	Reasoning
Built in place , not inside a <code>nexaris-outreach-agent/</code> wrapper	<code>03_PROJECT_STRUCTURE.md</code> shows a wrapper folder. Building in place keeps the planning documents beside the code instead of in a sibling directory. Everything below that level matches the spec exactly.
Tailwind v4 instead of v3	The structure document assumed v3, which needs <code>tailwind.config.js</code> and <code>postcss.config.js</code> . v4 runs as a Vite plugin and needs neither — theme settings live in <code>index.css</code> . Fewer files, current standard.
Loosened version pins to <code>>=</code>	Forced by an unresolvable conflict — see section 9.
runs treated as a 9th table	The schema creates it but the Phase 1 checklist lists only 8 tables. It is the data source for the Run Status panel and the finish time in the daily summary, so it is verified in Phase 1 alongside the others.
Build plan wins over spec §14	The two documents give slightly different orderings. The master prompt names the build plan authoritative. No requirement is dropped by either ordering.
Placeholder icons	The manifest referencing missing files would be a broken build. Generated a simple "N" mark to make it valid; replace with real branding in Phase 9.

9. Problems hit during the build, and how they were fixed

Problem 1 – Python dependencies could not be installed

Symptom: `ERROR: ResolutionImpossible` .

Cause: I had pinned every package to an exact version. But `supabase` and `anthropic` each require `httpx` within their own version range, and my exact `httpx` pin satisfied neither.

Fix: changed exact pins (`==`) to minimum pins (`>=`) and left `httpx` unpinned so pip can pick a version that satisfies both. Documented in the file itself so it is not "fixed" back later.

Problem 2 – `npm install` reported success but installed nothing

Symptom: exit code 0, empty output, no `node_modules` , and the build then failed on a missing package.

Cause: the first run was launched in the background and did not complete properly.

Fix: re-ran it in the foreground — 92 packages installed in 56 seconds. *Lesson worth keeping:* "the command exited 0" is not proof the work happened; I check the actual artifact.

Problem 3 – the build broke because of an unrelated project on your Desktop

Symptom: `@layer base`` is used but no matching `@tailwind base`` directive is present , with a stack trace pointing at `C:\Users\AUB\Desktop\node_modules\tailwindcss` .

Cause: this was the interesting one. When Vite starts it searches *upward* through parent folders for a PostCSS configuration. Your Desktop contains a leftover `package.json` , `postcss.config.js` , `tailwind.config.js` and `node_modules` from an earlier project using Tailwind **v3**. Our project sits inside the Desktop folder, so Vite walked up, found that v3 config, and tried to apply v3 processing to our v4 stylesheet.

Fix: declared an empty PostCSS config inside our own `vite.config.js` . That stops the upward search at our project boundary. I did **not** touch or delete anything on your Desktop.

Worth knowing: those stray files will keep interfering with any JavaScript project you create inside your Desktop folder. Cleaning them up eventually is a good idea — but that is your call, not mine to make.

Problem 4 – garbled characters in the agent output

Symptom: the self-check printed `AGENT  setup` .

Cause: the Windows console uses an older character encoding that cannot display the long dash character (—).

Fix: replaced the dashes in *printed output* with plain hyphens. Comments and documentation keep proper typography; only what reaches the console was changed.

10. What you should do right now — step by step

1 Test the agent

Open a terminal in the project folder and run:

```
agent/venv/Scripts/python.exe -m agent.main
```

Expect: the self-check box, with `[ok]` beside Python runtime and `[--]` beside the three credential lines. That is a pass — the three dashes are your to-do list, not errors.

2 Test the dashboard

```
cd dashboard
npm run dev
```

Open `http://localhost:5173`. **Expect:** a dark page titled "AI Outreach Agent", a setup status box with a green dot on Dashboard and a grey dot on Supabase, and a numbered list of the 10 sections coming in Phase 9.

Optional but worth doing: the terminal also prints a "Network" address like `http://192.168.1.x:5173`. Open that on your phone (same Wi-Fi) to confirm the mobile layout works. Stop the server with `Ctrl+C`.

2b If `npm run dev` fails on your machine

Try `cd dashboard` then `npm install` once, then `npm run dev` again. If it still fails, send me the error text — do not try to fix it yourself, it is more likely my problem than yours.

3 Push to GitHub

The commit is made and waiting. Add your remote once, then push:

```
git remote add origin https://github.com/YOUR-USERNAME/AiOutreachAgent.git
git push -u origin main
```

Replace `YOUR-USERNAME` with your actual GitHub username. After this first time, pushing is just `git push`.

I never run `git push` myself — you asked to be the one who decides what leaves your machine, and I have recorded that.

4 Create the Supabase project **BLOCKS PHASE 1**

1. Go to **supabase.com** and sign up (free — you are well within the free tier with 2 users).
2. Create a new project. Give it a name like `nexaris-outreach`.
3. Choose a database password and **save it somewhere safe** — it is shown once.
4. Pick the region closest to Lebanon (Frankfurt `eu-central-1` is usually the best latency).
5. Wait about two minutes for it to provision.
6. Go to **Project Settings** → **API** and copy three values:
 - Project URL
 - `anon` / `public` key
 - `service_role` key (click to reveal — this is the powerful one)

Send them to me in this chat, not into a file. Do not paste keys into any file yourself yet — I will put each one in the correct place. The `service_role` key must go only into `agent/.env`, never into the dashboard, because your repository is public.

5 Answer my open questions (as many as you can)

Listed in section 12. The two that matter soonest are the WhatsApp provider and whether you have a Claude API key yet. "I don't know" is a fine answer — I will recommend a default and flag it.

6 Tell me to start Phase 1

Once you have tested both apps and have the Supabase details, say the word. Phase 1 creates the database: I walk you through running the schema table by table, set up the security rules, build the data-access layer, and seed your default settings.

11. What you need before each future phase

Phase	What it builds	What you need ready
1	Database schema	Supabase project + all three keys
2	Account pool, sessions, health, scheduler	Nothing — proxies are not needed yet
3	LinkedIn + Instagram discovery	Login details for the 3 accounts
4	Claude analysis and scoring	Claude API key from console.anthropic.com, with a little credit
5	Message generation	Nothing new — but you review the messages
6	Approval workflow	Nothing
7	Sending and routing	WhatsApp Business API account (Twilio or 360dialog), or a decision on the informal method
8	CRM, follow-up, notifications	Your and Mohamad's WhatsApp numbers
9	The full dashboard	Real Nexaris icon/branding if you want it
10	Deploy and warm-up	3 sticky residential proxy IPs, a server (Oracle free tier or ~\$5 Hetzner VPS), a Vercel account

You do **not** need proxies or a server until Phase 10. Everything before that is built and tested locally at low volume, which is both free and safer for your accounts.

12. Open questions I need answers to

ID	Question	Needed by	What I will assume otherwise
Q1	WhatsApp sending: Twilio, 360dialog, or the informal method?	Phase 7	Build behind a swappable interface; Twilio as the reference
Q2	Same provider for notifications as for sending?	Phase 8	Same provider, separate configuration
Q3	How should LinkedIn replies be detected?	Phase 8	Check the inbox on each daily run
Q4	Should any alert go to only one number?	Phase 8	All five to both, except the approval reminder to Mohamad only
Q5	Which niche and country for the first real run?	Phase 10	Left blank for you to set from the dashboard
Q6	Is the Supabase project created?	Phase 1	Blocking — cannot proceed without it
Q7	Do you have a Claude API key?	Phase 4	Blocking at Phase 4